

레거시 시스템 AS-IS 분석 — 리버스 엔지니어링

소통나눔방 차세대 전환 사전 분석 · 포트폴리오 · 김재연

* 본 문서는 공공기관 실시스템의 내부 정보(서버 경로·계정·스키마·세션 구조·구체 수치 및 보안 취약점 상세)를 일절 포함하지 않습니다. 수행한 분석의 방법론과 도출 결과만 추상화하여 정리한 포트폴리오용 문서입니다.

배경과 문제

전환 대상 레거시(ASP 기반 공공 게시판 시스템)에는 요구사항정의서가 존재하지 않았습니다. 즉, '무엇을 만들어야 하는가'의 기준선 자체가 없는 상태에서 차세대(전자정부프레임워크/Spring) 전환을 준비해야 했습니다. 저는 이 공백을 메우기 위해, 남아있는 소스코드 전수조사를 통해 시스템을 역으로 분석하고 전환 요구사항의 기준선을 직접 구성했습니다.

분석 방법론

네 단계의 산출물을 순차적으로 만들어 서로 연결했습니다. 각 단계가 다음 단계의 입력이 되도록 설계해, 분석이 누락 없이 전환 계획으로 이어지게 했습니다.

단계	산출물	핵심 내용
① 인벤토리	AS-IS 인벤토리	화면·기능·데이터·배치를 전수 목록화하고 화면설계서와 상호 매핑. '분석 완료' 여부를 체크하며 진행률 관리
② 갭분석	표준 매핑 갭분석	레거시 데이터 구조를 전자정부 표준 모델과 대조하여 대응 유형(이름만 다름 / 구조 다름 / 한쪽에만 있음)으로 분류
③ 연동조사	외부연동·숨은요소 조사	기능분해도에 드러나지 않는 외부연동·인증·배치·파일처리를 키워드 전수검색으로 발굴하고 전환 위험도(상/중/하) 산정
④ 재배치	계층 재배치 설계	각 처리 로직을 Controller/Service/Mapper/JSP 및 공통 계층으로 재배치하는 기준 수립

① AS-IS 인벤토리 — 전수 목록화

공동 내비게이션에서 전체 메뉴 트리를 추출해 분석 대상 화면의 전체 지도를 먼저 확보했습니다. 이후 화면·기능을 하나씩 목록화하면서, 각 화면을 (a) 화면설계서, (b) 기존 ASP 소스, (c) 사용 데이터와 상호 매핑했습니다.

도출 결과

- 화면·기능 인벤토리 — 화면 ID ↔ 레거시 소스 ↔ 사용 데이터의 3자 매핑표
- 데이터 인벤토리 — 약 60 개 데이터 객체를 목록화하고 이관/폐기 후보를 1 차 판별
- 폐기 후보 식별 — 화면설계서에 없거나 실사용 흔적이 없는 화면·데이터를 '폐기 검토' 대상으로 표시
- 숨은 요소 조사 체크리스트 — 기능분해도 밖에 존재하는 요소를 찾기 위한 키워드 기반 점검 목록 작성

② 표준 매핑 갭분석

레거시가 '게시판을 데이터로 동적 정의'하는 독자 구조를 갖고 있어, 전자정부 표준 게시판 모델과 1:1 로 대응되지 않았습니다. 항목별로 표준과 대조해 대응 유형을 분류하고, 각 유형에 필요한 작업을 명시했습니다.

대응 유형	의미	전환 작업
① 이름만 다름	컬럼이 1:1 대응	매핑 설정에서 명칭만 교체
② 구조 다름	정규화·타입·의미가 상이	데이터 가공 및 로직 재구현
③ 한쪽에만 있음	대응 대상 자체가 없음	추가 구현 또는 폐기 판단
확인 필요	의미·용도 불명확	현행 소스 확인 후 분류 확정

특히 표준에 대응 개념이 없는 '레거시 전용 확장 기능'(게시판별 동적 확장필드·부가기능 플러그 등)을 ③ 유형으로 식별해, 유지 시 별도 구현이 필요함을 사전에 못 박았습니다. 이는 전환 공수 산정의 핵심 근거가 되었습니다.

③ 외부연동·숨은요소 조사 (가장 임팩트 큰 분석)

기능분해도만으로는 드러나지 않는 요소들이 전환의 실제 리스크였습니다. 소스 전수검색으로 외부연동·인증·배치·파일처리·보안 관련 패턴을 발굴하고, 각 항목을 전환 영향과 위험도로 정리했습니다.

발굴한 전환 리스크 유형 (추상화)

- 상용 컴포넌트 의존 — 행정전자서명(전자서명 연동)·상용 파일업로드 컴포넌트 등, 라이선스 승계 또는 대체 개발이 필요한 요소를 식별
- 다중 데이터소스 — 주 DB 외에 별도 목적의 데이터소스가 복수 존재함을 확인하고 이관 범위 재정의
- 외부 연동 — 메시징·문자·디렉터리 서비스 등 외부 API 연동을 발굴하고, 폐쇄망 환경에서의 호출 가능 여부를 확인 과제로 제기
- 인증·권한 구조 — 권한이 여러 세션 요소에 분산된 구조임을 파악하고, 표준 권한/롤 체계로의 재설계가 필요함을 도출
- 배치·스케줄 — 실행주기 기반 배치를 식별하고 표준 스케줄러로 재구현 대상으로 분류
- 보안 관점 리스크 — 시큐어코딩 진단에서 지적될 수 있는 위험 패턴을 사전 식별하여 신규 구축 시 조치 항목으로 등록

산출 방식

각 발견 항목을 '발견 근거 → 현재 방식 → 전환 시 과제 → 확인 필요사항'의 구조로 정리하고 위험도(상/중/하)를 부여해, 발주처·고도화팀과 협의할 수 있는 조치 목록으로 만들었습니다. '죽은 코드(미사용)'로 판별한 항목은 이관 대상에서 제외해 불필요한 공수를 줄였습니다.

④ 계층 재배포 설계 (AS-IS → Spring)

레거시의 화면 내 혼재된 처리 로직을, 표준 MVC 구조의 어느 계층으로 옮길지 요소 단위로 설계했습니다. 각 처리 단계를 Controller / Service / Mapper / JSP / 공통 중 하나로 배치하고, 전환 시 주의할 안티패턴을 함께 표시했습니다.

대표 사례 — 게시글 저장 로직 재배포

- 폼 파싱: 상용 컴포넌트 의존 → 표준 파일업로드로 교체 (공통 계층)
 - 입력 검증: XSS 방어 로직 → 공통 보안 필터로 이관
 - 트랜잭션: 수동 트랜잭션 처리 → 선언적 트랜잭션(@Transactional)으로 전환 (서비스 계층)
 - 위험 패턴 식별: 저장 후 재조회로 키를 취득하던 방식을 표준 키 생성 방식으로 대체하도록 명시
- 이 재배포 기준표는 이후 실제 개발자가 화면별로 구현 위치를 판단하는 가이드로 쓸 수 있도록 설계했습니다.

성과 요약

- 요구사항정의서가 없는 레거시를, 소스 전수조사 기반 리버스 엔지니어링으로 전환 요구사항 기준선을 직접 구성
- 기능분해도 밖의 숨은 리스크(상용 컴포넌트·외부연동·인증구조·보안 패턴)를 사전 발굴하여 전환 노력·비용 폭증 위험을 통제
- 표준 모델 캡슐화로 전환 공수와 난이도를 유형화하여 산정 근거 마련
- 계층 재배포 기준을 세워 분석 결과를 실제 구현 가이드로 연결

이 분석이 보여주는 역량

- 문서가 없는 시스템을 코드로부터 역으로 읽어내는 리버스 엔지니어링 능력
- 표면(기능분해도) 아래의 숨은 의존성·리스크를 찾아내는 분석 깊이
- 발견을 위험도·조치 목록으로 구조화하여 이해관계자와 협의 가능한 형태로 만드는 정리력
- 분석을 표준 아키텍처 설계·전환 계획으로 연결하는 개발 이해도

※ 실제 분석 산출물(인벤토리·캡슐화·연동조사·재배포 설계)은 시스템 내부 정보를 포함하므로 외부 제출용에서 제외했습니다. 본 문서는 그 방법론과 결과만 추상화한 것입니다.